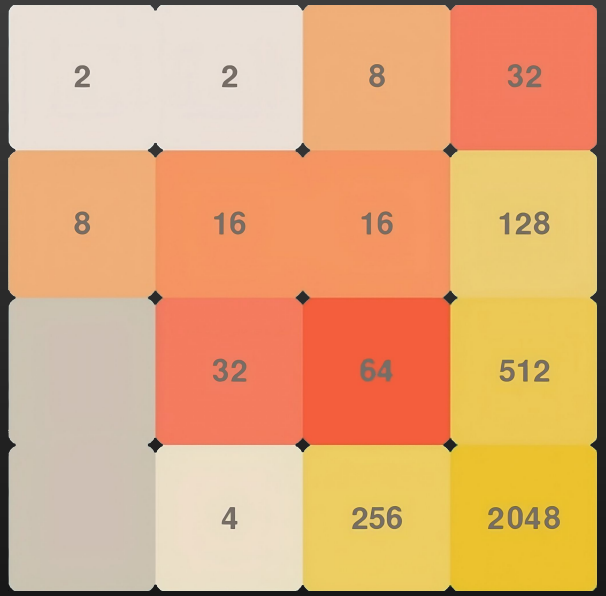
2048



Expectimax DLS Agent

Hayden Hawley

The Environment

Puzzle game on a 4x4 grid. The player slides numbered tiles (all powers of 2) by swiping up, down, left, or right. After each swipe, a new tile (2 or 4) appears in a random empty cell.

When two tiles of equal value collide, they merge into a tile equal to their sum and that value is added to the score.

Objective: Reach the 2048 tile and maximize high score.

PEAS Assessment

Performance Measure: Final game score

Environment: 4x4 grid with stochastic new tile placements

Actuators: Swipes in four directions (cannot swipe in a certain direction if all tiles are impeded by the wall or other tiles in that direction)

Sensors: Full board state (but not future tile location or value)

Agent Design

Began with a simple greedy search to get a feel for the environment and look for helpful heuristics.

Final iteration used depth-limited search and the expectimax algorithm to handle randomness. A swipe is represented by a choice node, and the location and value of a new tile is represented by a chance node.

Settled on a search depth of 8 for solid results within a reasonable time frame. The agent clones game states to simulate moves without altering the actual game.

Heuristics

Score: maximize total game score

Filled Cell Penalty: grows exponentially as the grid fills, beginning with a half-filled board. Helpful for freeing space near the endgame.

Max Tile Value: generate the highest possible single tile.

Domino Effect: favors rows or columns with uniformly increasing/decreasing tile values, sets up cascading merges.

Smoothness: penalizes large differences between neighboring tiles.

Game Over Penalty: big penalty to avoid terminal states (game over)

```
filled_penalty_weight = 0.05
max_tile_weight = 1.5
domino_weight = 0.1
smoothness_weight = 0.1
score_weight = 0.025

# Define the threshold after which penalty grows exponentially
threshold = total_cells // 2  # Half-filled board
if filled_cells > threshold:
    penalty_exponent = filled_cells - threshold
    filled_penalty = filled_penalty_weight * (2 ** penalty_exponent)
else:
    filled_penalty = 0

# Total heuristic score
total_score = (max_tile_weight * np.log2(max_tile) +
               domino_weight * domino -
               smoothness_weight * smoothness -
               filled_penalty +
               score_weight)
```

Testing and Tuning

Conducted 100 simulation runs for each agent on random game instances. Tracked highest score, average score, and standard deviation (used this to calculate coefficient of variation).

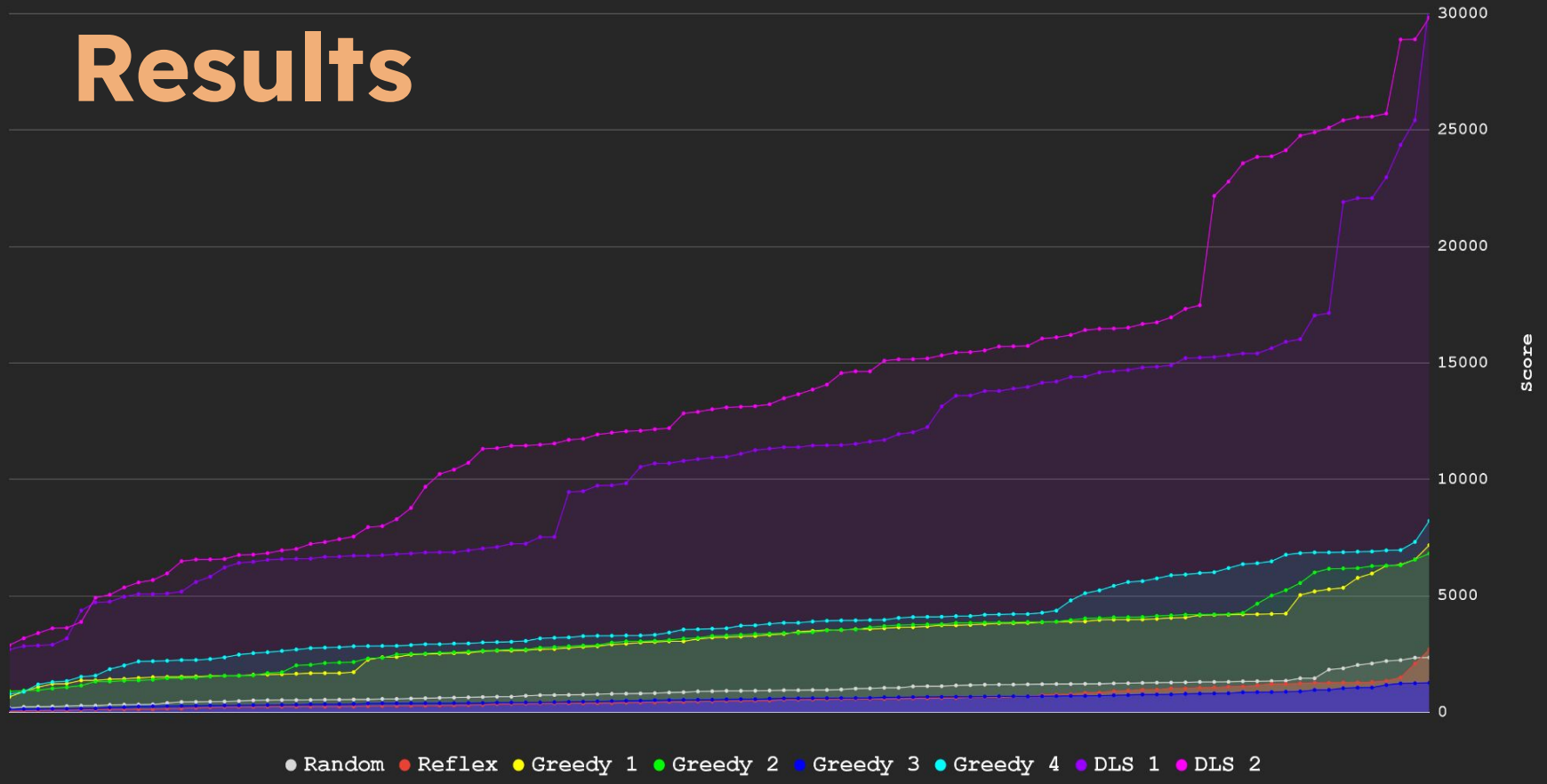
```
Agent: dls_agent
Number of runs: 100
Highest score: 28672
Average final score: 10330.04
Standard deviation: 5631.97
```

When I hit a plateau, I ran the agent in human mode for manual review. Looked for weaknesses and came up with new heuristics.

Results

	Random	Reflex	Greedy 1	Greedy 2	Greedy 3	Greedy 4	DLS 1	DLS 2	
Heuristic weights							2	8	search depth
			0.04	0.04	0.04	0.04	0.04	0.05	filled tiles
			1	1.5	1.5	1.5	1.5	1.5	max tile value
			1	1	0.0001	0.1	0.1	0.1	domino
			0.1	0.1	0.1	0.1	0.1	0.1	smoothness
								0.025	score
Highest Score	2352	2696	7176	6816	1260	8216	30500	29824	
Average Score	936	587	3150	3252	559	3913	10985	13454	
Standard Dev	489	451	1347	1428	264	1665	5344	6537	
CV (Avg / Std Dev)	1.91	1.30	2.34	2.28	2.12	2.35	2.06	2.06	

Results


30000
25000
20000
15000
10000
5000
0
Score
Random
Reflex
Greedy 1
Greedy 2
Greedy 3
Greedy 4
DLS 1
DLS 2

Analysis

Depth-limited search agents outperformed greedy agents by a huge margin. Expectimax played a big role in handling the game's randomness.

Even then, results varied drastically. I began to wonder if 2048 is a "solvable" environment or if it's possible to encounter an unwinnable game by pure chance.

Changes in heuristic weights became *much* more sensitive as search depth increased.

Potential Improvements

- Implement adaptive search depths based on the game state complexity.
- Adapt heuristic weights adjust dynamically depending on the game state.
- Experiment with reinforcement learning.
- Find more performance optimizations to let me increase search depth without sacrificing time.